

OPERATING SYSTEMS

Lab 10

Description. In this lab, you will learn how to use a dynamic library (*.so) in GNU/Linux and perform dynamic loading.

Task You will write a simple C program that performs the addition and subtraction of two integer numbers. For this, you will create two functions, one to perform addition and one to perform subtraction. A library that gets loaded at runtime should provide the two functions. You will then need to create a dynamic library (a.k.a., shared library) — a file with extension *.so.

(1) Start by creating a simple C-program (e.g., `lab10.c`) that reads two input integers from the keyboard and then invokes two functions `add(a,b)` and `sub(a,b)`. The main display the returned results. (2) The main file should include the standard static libraries (i.e., `stdio.h` and `stdlib.h`) for standard input and output, as well as a header file (e.g., `aslab.h`) that defines the prototypes of the two functions `add(a,b)` and `sub(a,b)`. (3) Create a new file, named `add1.c` that contains the implementation of the addition function. Then, compile it into an object file using `gcc -c add1.c -fpic`. (4) Create another file, named `sub1.c` that contains the implementation of the subtraction function and then compile it library into an object file using `gcc -c sub1.c -fpic`. (5) Create the shared “dynamic” library out of the previously generated two object files: `gcc *.o -shared -o dylib.so`. (6) Copy the library into the standard library directory (`cp dylib.so /usr/lib/dylib.so`). (7) Run `sudo ldconfig` to refresh the list of dynamic libraries. (8) Compile your main file into an object file while specifying the directory where GCC looks for header files (option `-I`). (7) Invoke the linker to link the library to the main program: `gcc -o main main.o -L. -dylib.so`. (8) Check that the linker has found all libraries using `ldd main`, where `main` is your main compiled file. Run your executable main file.

Instead of copying the dynamic library that you have created into the standard library directory, you could have added the complete path of your library to the library path variable, i.e., `LD_LIBRARY_PATH`. Then, you may have to run `ldconfig`.

You can use the `ps` command with options `-au` to display the running processes along with their virtual space size and real physical space size. These two pieces of information can be found under `VSZ` (Virtual Memory Size) and `RSS` (Resident Set Size), respectively. `RSS` indicates how much memory is currently allocated to a given process (swapped parts are not counted). `VSZ` includes all memory that the process can access.